

India Cancer Patient Dataset

Machine Learning Classification Step-by-Step Study Guide

Author: Nishant Soni | Date: June 20, 2026

Dataset: Kaggle — [ashyou09/india-cancer-patient-dataset-2022-2025](#)

Project Summary

Binary classification task: predict patient Status (Alive / Deceased) using clinical and demographic features. The dataset has 100,000 patients across 12 columns. Survival_Months was excluded as data leakage. Six models were compared using 5-fold cross-validation ranked by Weighted F1. The top 3 were hyperparameter-tuned with RandomizedSearchCV. Final evaluation uses a held-out test set that was never touched during model selection.

Table of Contents

- Chunk 1** Exploratory Data Analysis (EDA) - Basic
- Chunk 1.5** Deep EDA - Feature vs Target
- Chunk 2** Data Preprocessing
- Chunk 3** Pipeline with OneHotEncoder + StandardScaler
- Chunk 4** Model Comparison with Cross-Validation
- Chunk 5** Hyperparameter Tuning with RandomizedSearchCV
- Chunk 6** Final Evaluation on Test Set
- Appendix** Key Concepts Quick Reference

Chunk 1 - Exploratory Data Analysis (EDA)

Before building any model, we explore the raw data to understand its shape, data types, missing values, and class distribution. This step catches problems early and builds intuition about which features might matter.

1.1 Load and Inspect the Data

```
import pandas as pd

df = pd.read_csv("india_cancer_patients_2022_2025.csv")

df.shape          # (100000, 12) – 100k rows, 12 columns
df.head()         # preview first 5 rows
df.T              # transpose: see all column names vertically
```

Transposing with `df.T` is useful when there are many columns — it lets you read column names vertically without horizontal scrolling.

1.2 Check Class Balance

```
df["Status"].value_counts(normalize=True)

# Deceased    0.636  (64%)
# Alive       0.364  (36%)
```

The target is **imbalanced** — 64% Deceased, 36% Alive. A model that always predicts Deceased gets 64% accuracy without learning anything. We use **Weighted F1 score** as our primary metric instead.

1.3 Missing Values and Duplicates

```
df.isna().sum()          # all zero – no missing values

df.drop(columns="Patient_ID").duplicated().sum() # 19 real duplicates found
```

NOTE: Always exclude ID columns when checking duplicates

`Patient_ID` is unique per row by design, so including it in `.duplicated()` makes every row appear unique — you will always get 0. Drop ID columns first. The Kaggle reference notebook made this mistake and reported 0 duplicates incorrectly.

1.4 Feature Distributions

```
for col in ["Gender", "Cancer_Type", "Stage", "Treatment_Type"]:
    print(df[col].value_counts())

# Gender:          Female 67k, Male 33k
# Cancer_Type:     9 types – Breast Cancer most common (28k)
# Stage:           Stage III most common (35k)
# Treatment_Type:  7 types – Palliative Care most common (21k)
```

WARNING: Survival_Months is DATA LEAKAGE — excluded from all models

Survival_Months records how long the patient survived AFTER diagnosis. For deceased patients this directly encodes the time-to-death outcome. A model trained with this column is cheating: it learns the answer, not the features. A Kaggle notebook that kept this column reported 97% accuracy — entirely artificial. We exclude it.

Chunk 1.5 - Deep EDA: Feature vs Target

The key question: does each feature actually separate Alive from Deceased? A feature with no separation power will not help the model, no matter how complex the algorithm.

1.5.1 Numerical Features — Boxplots by Status

```
import seaborn as sns

sns.boxplot(data=df, x="Status", y="Age",
            palette={"Alive": "steelblue", "Deceased": "salmon"})
```

Age shows **minimal separation** between groups. The medians are similar, meaning age alone is a weak predictor. It may still contribute when combined with cancer type and stage.

1.5.2 Mortality Rate by Category (correct approach)

```
# Use rates (deceased / total), NOT raw counts

# Raw counts mislead when category sizes differ

deceased = df[df["Status"] == "Deceased"].groupby("Stage").size()

total     = df.groupby("Stage").size()

stage_mortality = (deceased / total * 100)
```

1.5.3 Correlation with Target

```
# numeric_only=True silences a FutureWarning in pandas 2.0+

corr = df_encoded.corr(numeric_only=True)["Status"].drop("Status")

corr.sort_values(key=abs, ascending=False)

# Stage_Stage IV and Cancer_Type_Lung Cancer appear near the top
```

1.5.4 Why Scaling Is Needed for Logistic Regression

Age ranges 1-95. One-hot dummy columns are 0 or 1. If fed raw to Logistic Regression the weight for Age is on a completely different scale than the dummy weights — making optimisation unstable. StandardScaler brings all features to mean=0, std=1.

NOTE: Tree models do NOT need scaling

Random Forest, XGBoost, and Gradient Boosting split on thresholds, not magnitudes. Scaling has zero effect on tree models. Only distance-based (KNN, SVM) and linear gradient-based models (Logistic Regression, Neural Networks) require it.

Chunk 2 - Data Preprocessing

Step 1 — Remove Duplicates and Drop Patient_ID

```
df_clean = df.drop(columns="Patient_ID").drop_duplicates()

# 100,000 rows -> 99,981 rows (19 duplicates removed)
```

Patient_ID is just an identifier with no predictive signal. If kept, a model might memorise IDs rather than learn real patterns. We also remove the 19 duplicate records.

Step 2 — Encode the Target Variable

```
df_clean["Status"] = df_clean["Status"].map({"Alive": 0, "Deceased": 1})

# Alive=0, Deceased=1

# Convention: 1 = the positive class (the event we are predicting)
```

All sklearn models require numeric targets. Setting Deceased=1 means all metrics (precision, recall, F1) treat 'Deceased' as the positive class by default.

Step 3 — Drop Leakage and Redundant Columns

```
# Survival_Months -> LEAKAGE: outcome cannot be known at diagnosis time
# State / City -> REDUNDANT: Hospital_Name already captures location
# Diagnosis_Date -> needs feature engineering (year/month extraction)
# Patient_ID -> already dropped in Step 1
```

Step 4 — Train / Test Split

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,      # 80% train, 20% test
    random_state=42,    # reproducible results
    stratify=y          # preserve class ratio in both splits
)

# X_train: 79,984 rows   X_test: 19,997 rows
```

NOTE: Why stratify=y?

Without stratify, a random split might put more Deceased patients in one set by chance. stratify=y forces both train and test to maintain the same 64/36 class ratio as the full dataset, making evaluation fair and consistent.

Chunk 3 - Pipeline with OneHotEncoder + StandardScaler

Why Pipeline Instead of pd.get_dummies?

Aspect	pd.get_dummies	Pipeline + OneHotEncoder
Encoding happens	Before CV — outside folds (leakage risk)	Inside each CV fold (correct)
Unseen categories	Silently produces wrong output shape	handle_unknown='ignore' is safe
Reusable on new data	Must re-run entire notebook	pipeline.predict(new_data) works
Works with GridSearchCV	No	Yes

Building the ColumnTransformer

ColumnTransformer applies different transformations to different columns simultaneously, then concatenates the results into one feature matrix.

```
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline

num_cols = ["Age"]
cat_cols = ["Gender", "Cancer_Type", "Stage", "Treatment_Type", "Hospital_Name"]

preprocessor = ColumnTransformer(transformers=[
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore",
                          sparse_output=False), cat_cols),
])
```

Wrapping into a Pipeline

```
pipeline = Pipeline(steps=[
    ("preprocessor", preprocessor), # Step 1: encode + scale
    ("classifier", some_model),    # Step 2: train the model
])

# One call handles everything:
pipeline.fit(X_train, y_train)
pipeline.predict(X_test)
```

NOTE: Pipeline hyperparameter naming: stepname__parametername

When tuning hyperparameters inside a Pipeline, prefix with the step name and two underscores. Our step is named 'classifier', so use: classifier__n_estimators, classifier__max_depth, classifier__C, etc.

Chunk 4 - Model Comparison with Cross-Validation

What Is 5-Fold Cross-Validation?

Instead of evaluating on one fixed split (which can get lucky or unlucky), we divide training data into 5 equal parts. The model trains on 4 parts and is scored on the 5th, rotating 5 times. The final score is the mean of all 5 runs.

```
Fold 1: [TEST ][TRAIN][TRAIN][TRAIN][TRAIN] -> score 1
Fold 2: [TRAIN][TEST ][TRAIN][TRAIN][TRAIN] -> score 2
Fold 3: [TRAIN][TRAIN][TEST ][TRAIN][TRAIN] -> score 3
Fold 4: [TRAIN][TRAIN][TRAIN][TEST ][TRAIN] -> score 4
Fold 5: [TRAIN][TRAIN][TRAIN][TRAIN][TEST ] -> score 5

Final = mean(score 1 ... score 5)
```

Cross-Validation Code

```
scores = cross_val_score(
    pipeline, X_train, y_train,
    cv=5,                # 5 folds
    scoring="f1_weighted", # weighted F1 across both classes
    n_jobs=-1            # use all CPU cores in parallel
)

print(f"F1 = {scores.mean():.4f} +/- {scores.std():.4f}")
```

Models Compared (KNN and SVM skipped — too slow on 100k rows)

Model	Type	Scaling Needed?	Speed
Logistic Regression	Linear classifier	Yes	Very fast
Decision Tree	Single decision tree	No	Fast
Random Forest	Parallel ensemble of trees	No	Fast
Extra Trees	More random parallel trees	No	Fast
Gradient Boosting	Sequential ensemble	No	Slow
XGBoost	Optimised sequential trees	No	Fast

Results with Default Hyperparameters

Rank	Model	Weighted F1	F1 Std	Comment
1	Gradient Boosting	0.7155	0.0039	Sequential trees — top performer
2	Logistic Regression	0.7150	0.0025	Linear, stable, nearly as good

3	XGBoost	0.7110	0.0023	Optimised GB, fast and stable
4	Random Forest	0.6532	0.0020	Lower here, parallel trees
5	Extra Trees	0.6473	0.0016	Most stable but lower F1
6	Decision Tree	0.6397	0.0045	Simplest — most unstable

NOTE: What F1 Std tells you

A high std (e.g. 0.004) means performance varies a lot between folds — the model is sensitive to which data it sees.
A low std means consistent generalisation. When two models have similar mean F1, prefer the one with lower std.

Chunk 5 - Hyperparameter Tuning with RandomizedSearchCV

GridSearchCV vs RandomizedSearchCV

Aspect	GridSearchCV	RandomizedSearchCV
Strategy	Every combination in the grid	Randomly sample n_iter combinations
Example	5x4x3 = 60 combos x 5 folds = 300 fits	20 combos x 5 folds = 100 fits
Time	Exponential with grid size	Controlled via n_iter parameter
Result	Guaranteed grid-optimal	Near-optimal (research-backed)
Best for	Small grids, fast models	Large grids, slow models like GB

loguniform — Better Sampling for Parameters Spanning Orders of Magnitude

For Logistic Regression's C parameter (0.001 to 100), uniform sampling wastes most draws in the 50-100 range. loguniform samples on a log scale, giving equal probability to each order of magnitude: 0.001-0.01, 0.01-0.1, 0.1-1, 1-10, 10-100.

```
from scipy.stats import loguniform

lr_params = {
    "classifier__C":      loguniform(0.001, 100), # log-scale sampling
    "classifier__solver": ["lbfgs", "saga"],
    "classifier__penalty": ["l2"],
}
```

XGBoost — Key Hyperparameters

Parameter	What it controls
n_estimators	Number of trees. More = better fit but slower, risk of overfit.
learning_rate	Step size per tree. Lower = more robust, needs more trees.
max_depth	Tree depth. Deeper = more complex patterns but higher overfit risk.
subsample	Fraction of rows per tree (row sampling). Adds randomness.
colsample_bytree	Fraction of features per tree. Also reduces overfit.
gamma	Min loss reduction to make a split. Higher = more conservative.
reg_alpha	L1 regularisation on leaf weights. Pushes some to exactly zero.

Gradient Boosting — Key Hyperparameters

Parameter	What it controls
-----------	------------------

n_estimators	Number of sequential trees.
learning_rate	Step size per tree correction. Pair low rate with more estimators.
max_depth	Default is 3 (shallow). Shallow trees work well with many boosting rounds.
min_samples_split	Min samples to split a node. Higher = simpler, more conservative trees.
subsample	Row fraction per tree. Values below 1.0 = stochastic gradient boosting.

NOTE: best_estimator_ is already retrained on full X_train

After RandomizedSearchCV finishes, best_estimator_ is the full Pipeline with best hyperparameters, retrained on all of X_train. Call .predict(X_test) directly.

WARNING: Gradient Boosting time warning — expect 5 to 10 minutes

sklearn GradientBoostingClassifier has no GPU support. With n_iter=20 and cv=5 on 80k rows, it performs 100 full fits. Each takes 3-5 seconds. XGBoost is significantly faster for the same configuration.

Chunk 6 - Final Evaluation on Held-Out Test Set

The Golden Rule of Test Set Usage

Throughout Chunks 4 and 5, only `X_train` was used (inside cross-validation). `X_test` was completely untouched. We evaluate on it exactly once, at the very end. If you peek at the test set multiple times to select models or tune parameters, you effectively overfit to it and the final score is no longer trustworthy.

Metrics Explained

Metric	Meaning	When to prioritise
Accuracy	Correct predictions / Total	Only for balanced classes
Precision	$TP / (TP + FP)$: of all predicted Deceased, how many really are?	When false alarms are costly
Recall	$TP / (TP + FN)$: of all actual Deceased, how many did we catch?	When missed detections are costly
F1	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	Balance of both
Weighted F1	F1 per class weighted by class count	Our dataset — 64/36 imbalance
ROC-AUC	Area under TPR vs FPR curve. 0.5 = random, 1.0 = perfect	Ranking quality across thresholds

Confusion Matrix Layout

	Predicted: Alive	Predicted: Deceased
Actual: Alive	True Negative	False Positive
Actual: Deceased	False Negative	True Positive

False Negative (FN) = model said Alive but patient is Deceased
-> most dangerous clinical error: missed detection

False Positive (FP) = model said Deceased but patient is Alive
-> less critical: unnecessary follow-up

Final Evaluation Code

```
tuned_models = {  
    "Logistic Regression": lr_search.best_estimator_,  
    "XGBoost": xgb_search.best_estimator_,  
    "Gradient Boosting": gb_search.best_estimator_,  
}  
  
for name, pipeline in tuned_models.items():  
    y_pred = pipeline.predict(X_test)
```

```
y_prob = pipeline.predict_proba(X_test)[: , 1] # P(Deceased)

print(accuracy_score(y_test, y_pred))

print(f1_score(y_test, y_pred, average="weighted"))

print(roc_auc_score(y_test, y_prob))

print(classification_report(y_test, y_pred,
                             target_names=["Alive", "Deceased"]))
```

NOTE: ROC-AUC vs Weighted F1

ROC-AUC measures ranking quality across ALL probability thresholds, not just 0.5. In a clinical setting you might lower the threshold (e.g. to 0.3) to catch more Deceased patients at the cost of more false alarms. The ROC curve visualises this trade-off across every possible threshold. Weighted F1 only measures performance at the default 0.5 threshold.

Appendix - Key Concepts Quick Reference

Data Leakage

Leakage occurs when information unavailable at prediction time is included during training. The model learns to cheat rather than generalise. Leakage produces inflated scores that collapse on real new data. In this project: Survival_Months is leakage because you cannot know how long a patient survives at the time of diagnosis.

Dummy Variable Trap

When one-hot encoding N categories, you only need N-1 columns. The Nth is implied when all others are 0. Including all N creates perfect multicollinearity, which destabilises Logistic Regression. Use `drop_first=True` in `get_dummies`, or `drop='first'` in `OneHotEncoder`. The first alphabetical category becomes the baseline.

Overfitting vs Underfitting

Condition	Train F1	Test F1	Diagnosis	Fix
Overfitting	High (0.95+)	Low (0.70)	Memorised training data	Regularise, reduce features, more data
Underfitting	Low (0.65)	Low (0.65)	Too simple for the task	More features, deeper model
Good fit	High (0.80)	High (0.78)	Generalises well	Ready to evaluate / deploy

Gradient Boosting vs XGBoost vs Random Forest

Property	Random Forest	Gradient Boosting	XGBoost
Tree building	Parallel independent trees	Sequential (each corrects previous)	Sequential (optimised C++)
Speed	Fast	Slow	Fast
Overfit risk	Low (averaging)	Medium (can chase noise)	Low (regularisation built-in)
Hyperparams	Less sensitive	Very sensitive to <code>lr</code> + <code>n_estimators</code>	Moderate
Scaling needed	No	No	No

When to Use Each Metric

Metric	Use when...
Accuracy	Classes are balanced (50/50)
Precision	False positives are costly — e.g. spam filter wrongly blocks good emails

Recall	False negatives are costly — e.g. missing a cancer diagnosis
F1	You want a balanced trade-off between precision and recall
Weighted F1	Classes are imbalanced — our dataset has a 64/36 split
ROC-AUC	You want to evaluate ranking quality across all decision thresholds

Full Project Workflow at a Glance

Chunk	Goal	Key Tool
1	Load and inspect raw data	pandas: shape, value_counts, isna, duplicated
1.5	Visualise each feature vs the target	seaborn boxplot, bar chart, corr()
2	Clean, encode target, train/test split	drop_duplicates, map(), train_test_split
3	Build proper preprocessing pipeline	ColumnTransformer, OneHotEncoder, StandardScaler
4	Compare 6 models by Weighted F1	cross_val_score with cv=5, f1_weighted
5	Tune top 3 models	RandomizedSearchCV, loguniform, n_iter=20-50
6	Final evaluation on unseen test set (once)	classification_report, ConfusionMatrix, ROC-AUC